Vision-Language Geometric Programming for Long-Horizon Robotic Assembly

Michael Shell, *Member, IEEE,* John Doe, *Fellow, OSA,* and Jane Doe, *Life Fellow, IEEE*

***Abstract*—The abstract goes here.**

***Index Terms*—IEEE, IEEEtran, journal, LaTeX, paper, template.**

I. Introduction

THIS demo file is intended to serve as a "starter file" for IEEE journal papers produced under LaTeX using IEEEtran.cls version 1.8b and later.

II. Related Work

III. Preliminaries and Problem Statement

Traditionally, robotic task execution relied heavily on hard-coded trajectories, which severely limited system generalization. The advent of Task and Motion Planning (TAMP) significantly lowered the deployment barrier by abstracting continuous motion into symbolic predicate logic, such as the Planning Domain Definition Language (PDDL) [11]. However, in long-horizon tasks, this framework remains strictly dependent on human experts to manually and exhaustively define all preconditions and effects. This zero-tolerance for logical incompleteness makes manual specification highly susceptible to critical omissions.

To alleviate this manual bottleneck, recent advancements have integrated Vision-Language Models (VLMs) as semantic front-ends to autonomously generate TAMP domain rules [8]. While this greatly enhances system usability, such pure semantic-driven architectures suffer from inherent logical brittleness. Because Large Language Models (LLMs) operate without underlying physical grounding, they are prone to logical discontinuities during long-horizon reasoning. For instance, an LLM might generate a high-level command to "grasp the egg" while omitting the implicit physical prerequisite to "open the drawer" first. Without a geometric feedback mechanism, such semantic gaps inevitably lead to cascading execution failures.

Theoretically, Logic-Geometric Programming (LGP) [1] serves as the optimal paradigm to bridge this semantic-to-physical gap, demanding an even lower task specification threshold than VLM-guided TAMP. By tightly coupling discrete symbolic logic and continuous trajectory optimization into a single joint objective function, LGP only requires an abstract terminal goal (e.g., `(on pot egg)`). When the underlying continuous solver attempts to penetrate a closed drawer's collision mesh to reach the egg, the kinematic infeasibility naturally triggers geometric backtracking. Driven purely by rigorous mathematical optimization and spatial constraints rather than probabilistic guessing, the system autonomously deduces and inserts the required "open drawer" action sequence.

However, applying native LGP off-the-shelf to complex real-world tasks (e.g., cooking a meal involving hundreds of sequential steps) exposes two prohibitive bottlenecks:

1) Combinatorial Explosion of the Search Space: Blindly relying on geometric backtracking to derive long-horizon logic leads to a combinatorial explosion of the state space. The computational cost of evaluating deeply nested, kinematically invalid branches renders the solver intractable.

2) Singularities in Closed Kinematic Loops: Native LGP relies on a strict tree topology for its kinematic tree, defining rigid parent-child frame relationships. While many-to-one object allocations (e.g., placing multiple eggs into a single pot) maintain a valid closed logic, multi-support assemblies (e.g., placing a rigid toy-block roof across multiple supporting pillars) introduce closed kinematic loops. Forcing dynamic one-to-many topology switches in such scenarios directly violates the tree-based assumptions of the k-order Markov Optimization (KOMO) engine [2], leading to mathematical singularities and solver deadlocks.

IV. Method

Motivated by the aforementioned logical brittleness of pure semantic planners and the computational bottlenecks of native continuous solvers, we propose **VLM-LGP** (Vision-Language Model-guided Logic-Geometric Programming). It is a hierarchical planning framework that leverages a VLM to generate semantically meaningful and horizon-reducing subgoals, which in turn tightly guide an underlying LGP solver.

A. Phase 0: Scene Initialization and Semantic-to-Geometric Binding

To bridge the continuous visual observation space with the discrete symbolic domain required by the LGP solver, the pipeline aligns visual inputs with unlabeled physical proxies predefined in the simulation environment (e.g., an anonymous `.g` configuration). Specifically, the VLM recognizes object types and shapes from the visual workspace and matches them to these anonymous simulation entities. Concurrently, objects are spatially sorted based on a heuristic utilizing their radial

M. Shell was with the Department of Electrical and Computer Engineering, Georgia Institute of Technology, Atlanta, GA, 30332 USA e-mail: (see http://www.michaelshell.org/contact.html).

J. Doe and J. Doe are with Anonymous University.

Manuscript received April 19, 2005; revised August 26, 2015.

Euclidean distance from the robot base, establishing a preliminary reasoning sequence. While standard TAMP inherently demands a highly accurate virtual twin of the real world—and we defer real-time point-cloud-based dynamic scene generation to future work—this visual-to-geometric alignment provides a sufficient deterministic anchor for all subsequent execution.

B. Phase 1: Extraction of the Layered Precedence Graph

Listing 1. Example of the Layered Precedence Graph extracted by the VLM. This structured schema restricts the geometric search depth in downstream

```
{
  "assembly_nodes": [
    {
      "node_id": 1,
      "description": "Foundation",
      "actions": [
        { "object": "base", "placed_on"
            : "table", "at_slot": "
            place_base1" },
        { "object": "cylinder", "
            placed_on": "base", "at_slot
            ": "top_left" }
      ]
    },
    {
      "node_id": 2,
      "description": "Second Layer",
      "actions": [
        { "object": "base", "placed_on"
            : ["cylinder"], "at_slot":
            null },
        { "object": "cylinder", "
            placed_on": "base", "at_slot
            ": "mid_left" }
      ]
    }
  ]
}
```

In complex multi-part assembly tasks, restricting the state-space search via sequence-defining precedence graphs is an established methodology (e.g., as explored in systems like Fabrica [10]). While existing approaches often rely on dedicated physics-based disassembly planners, we propose an alternative semantic framework. Instead of requiring intensive kinematic pre-computations, a Vision-Language Model acts as a high-level zero-shot parser to process visual assembly instructions.

Similar to the sub-goal temporal decomposition seen in hierarchical planning [4], the VLM extracts a semantic *Layered Precedence Graph* $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ (see Listing 1). Each node $v \in \mathcal{V}$ encapsulates a disjoint assembly stage (e.g., placing the foundation before the second layer), generating an ordered sequence of semantic placement tasks. This graph serves as an abstract temporal skeleton for downstream processing.

C. Phase 2: Dynamic Resource Allocation and Batch Compilation

To overcome the combinatorial explosion identified in Section III, we introduce a dynamic pruning pipeline comprising a *Semantic Resource Allocator* and a *Logic Batch Compiler*. This phase bridges the global temporal skeleton with localized execution constraints.

For each specific task node extracted from the global Layered Precedence Graph in Phase 1, the Semantic Resource Allocator first actively binds generic object requests (e.g., "cylinder") to uniquely available physical entities in the live simulation inventory (e.g., `cyl1` or `cyl2`).

Crucially, instead of blindly outputting a single static plan, the system leverages the VLM's reasoning capabilities to explicitly generate multiple candidate logic sub-graphs (execution sequences) (see Fig. **??** for illustration) that satisfy the current node's structural requirements. The VLM then acts as an autonomous evaluator, assessing these deterministic routes against spatial heuristics—such as minimizing arm occlusion, avoiding redundant tool switches, or favoring top-to-bottom manipulation. It explicitly selects the most physically feasible trajectory and outputs the rationale behind its decision.

Only after this rigorous semantic filtering and logical validation is the selected optimal sequence passed to the Logic Batch Compiler. This compiler translates the chosen sub-graph into hardware-safe execution batches guided by strict dependency rules, explicitly outputting flat, atomic facts adhering to native LGP syntax (e.g., generating `(on top_left_base1 cyl1)` alongside designated action rules like `pick_cylinder`).

Consequently, by harnessing the VLM's physical understanding of the real world combined with LGP's unified continuous-discrete planning, this framework elevates the overall system robustness and success rate to a new level. Furthermore, benefiting from the rapid advancement of large language models, the success rate of the semantic reasoning phase is already exceptionally high, establishing a solid foundation for the performance comparisons presented in the experimental evaluations.

D. Phase 3: Constrained Geometric Execution

The geometric execution phase translates the logic bounds dynamically compiled in Phase 2 into continuous trajectory optimization using the native C++ LGP infrastructure [1]. Because the VLM has strictly pruned the logical permutations, the k-order Markov Optimization (KOMO) solver focuses solely on resolving local geometric feasibility. The core action skeleton is refined through three specialized, physics-grounded manipulation primitives.

1) Semantic-Guided Auxiliary Grasping (`action_pick`): To handle large-scale objects, standard LGP solvers often fail during inverse kinematics if targeting the ungraspable object's geometric centroid. We introduce a decoupled *Semantic Grasping Site* paradigm. During `action_pick`, the system traverses the object's kinematic tree and re-parents the optimization target to an auxiliary proxy frame, $targetF$ (e.g.,

a designated handle). The grasp pose leverages a 6-DOF free joint relative to the gripper:

$$f_{snap} \leftarrow \text{initFrameDof}(gripper, targetF) \tag{1}$$

This transforms a highly non-convex full-body collision problem into a tractable sub-problem focused tightly on the valid grasping manifold.

2) Topological Decoupling for Multi-Support (`action_place_multi`): Native LGP assumes strict open-chain kinematic trees, which fail in multi-support structural scenarios. We resolve this by introducing a highly adaptable unactuated 6-DOF *Virtual Anchor* (A_v) parented directly to the world frame.

For placing an object O onto a composite set of support frames $S_{1..n}$, A_v is assigned an initial pose matching the calculated target centroid. The solver then enforces a soft optimization constraint aligning A_v to the target world pose, while employing a rigid kinematic switch allocating ownership of object O to A_v:

$$\Phi_{anchor} = \|\phi_{\text{poseDiff}}(A_v, O, x_t)\|^2 \approx 0 \tag{2}$$

This manipulation elegantly breaks closed kinematic loops, granting the KOMO solver dynamic capability to iteratively interpolate the most stable geometric descent.

3) Trajectory Shaping and Three-Stage Safety Protocol: To overcome solver deadlocks caused by static long-horizon obstacles, all generated sequences are heavily regularized through dynamic trajectory shaping:

Pre-Action Funnels: To prevent lateral sliding collisions during delicate placements or insertions, we mathematically inject relative positional waypoints at $t - 0.5$ and $t - 0.1$. This constructs a virtual "funnel" forcing the payload precisely above the anchor with a strict vertical standoff distance (e.g., $d_z = 0.05\,\text{m}$):

$$\Phi_{\text{funnel}} = \|\phi_{\text{posRel}}(O, A_v, x_{t-0.1}) - \begin{bmatrix}0 & 0 & d_z\end{bmatrix}^T\|^2_{sos} \tag{3}$$

Three-Stage Phased Collision Relaxations: Furthermore, continuous collision constraints $FS_{negDistance}$ between the moving gripper parts and static obstacles are segmented temporally. During the *transit phase* ($t \in [t - 1.0, t - 0.2]$), strict safety margins ($d_{safe} \geq 0.03\,\text{m}$) are enforced utilizing bounding hierarchies. However, within the *final approach phase* ($t - 0.2$ to t), these margins are hierarchically relaxed to allow physical manifold contact, enabling deadlock-free grasping and precision placement.

E. Phase 4: State Verification and Safety Halting

Given the inherent uncertainties of physical execution (e.g., slipping, grasping failures), open-loop trajectory execution is risky for long-horizon assembly. To address this, we introduce a *Visual-Semantic Validator* that performs state verification after each executed batch.

The validator captures a top-down visual observation of the current workspace and utilizes computer vision, combined with the VLM, to detect geometric deviations against the expected blueprint state established in Phase 1. If an error is detected—such as a missing component in a designated slot or an unexpectedly placed object (an "intruder")—the system immediately halts the standard planning pipeline. While dynamic closed-loop replanning and autonomous physical recovery remain theoretical extensions for future work, this rigorous error detection mechanism effectively prevents minor execution anomalies from cascading into catastrophic structural failures, establishing a critical safety boundary for the system.

V. Experimental Results

A. Performance Evaluation

Experimental data and baseline comparisons with native LGP go here.

VI. Conclusion

The conclusion summarizes the contributions of VLM-LGP in long-horizon assembly tasks.

Appendix A
Proof of the First Zonklar Equation

Appendix one text goes here.

Acknowledgment

The authors would like to thank...

References

[1] M. Toussaint, "Logic-geometric programming for multi-step manipulation," in *Proceedings of the International Joint Conference on Artificial Intelligence (IJCAI)*, 2015.
[2] M. Toussaint, "Newton methods for k-order Markov optimization," *arXiv preprint arXiv:1407.0414*, 2014.
[3] J. Schoeler, F. T. Rehder, and M. Toussaint, "R-LGP: Reactive Logic-Geometric Programming," *arXiv preprint arXiv:2310.02791*, 2023.
[4] J. Doe et al., "Receding Horizon Hierarchical Logic-Geometric Programming," *arXiv preprint arXiv:2110.03420*, 2021.
[5] F. Lin et al., "Text2Motion: From Natural Language to Logic-Geometric Programming," *arXiv preprint arXiv:2303.12153*, 2023.
[6] General VLA Reference, "Vision-Language-Action Models for Robotics," *Placeholder*, 2024.
[7] H. Kopka and P. W. Daly, *A Guide to LaTeX*, 3rd ed., 1999.
[8] M. Toussaint et al., "General VLA Reference for Task and Motion Planning," *arXiv preprint arXiv:2410.02193*, 2024.
[9] J. Doe et al., "Vision-Language Models for Robotic Assembly," *arXiv preprint arXiv:2407.09829*, 2024.
[10] Y. Zhu et al., "Fabrica: Dual-Arm Assembly of General Multi-Part Objects via Integrated Planning and Learning," *arXiv preprint arXiv:2506.05168*, 2024.
[11] D. McDermott *et al.*, "PDDL-the planning domain definition language," *Technical Report, Yale Center for Computational Vision and Control*, 1998.